МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 4

Студент группы *ИВБО-20-24 Дьяков Андрей Геннадьевич* _____
(подпись)

Ассистент *Брайловский Андрей Валерьевич*

_____
(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

АНАЛИТИЧЕСКИЕ ЗАПРОСЫ: ОКОННЫЕ ФУНКЦИИ И ПОСТРОЕНИЕ СВОДНЫХ ТАБЛИЦ

Цель работы: Целью данной практической работы является формирование у студентов углубленных навыков работы со сложными аналитическими запросами в СУБД PostgreSQL.

Задание №1: использование ранжирующих функций

Листинг 1

```
-- Задание №1: Ранжирование заказов по сумме внутри каждого менеджера
WITH RankedOrders AS (
    SELECT
        -- Формируем ФИО менеджера из составного типа fio_type
        (e.full_name).last_name || ' ' || (e.full_name).first_name AS
manager_name,
        co.id AS order_id,
        co.total_amount,
        -- Ранжируем заказы по убыванию суммы в пределах каждого менеджера
        DENSE_RANK() OVER (PARTITION BY e.id ORDER BY co.total_amount DESC) AS
order_rank
    FROM employee e
    JOIN customer_order co ON e.id = co.employee_id
)
SELECT
    manager_name AS "Менеджер",
    order_id AS "ID Заказа",
    total_amount AS "Сумма заказа",
    order_rank AS "Ранг"
FROM RankedOrders
WHERE order_rank <= 3 -- Оставляем только топ-3
ORDER BY manager_name, order_rank;
```

	Менеджер	ID Заказа	Сумма заказа	Ранг
1	Лебедева Екатерина	3	45 000	1
2	Лебедева Екатерина	7	0	2
3	Новиков Игорь	4	210 000	1
4	Новиков Игорь	8	32 000	2
5	Попов Александр	9	183 000	1
6	Попов Александр	2	120 000	2
7	Попов Александр	6	99 000	3
8	Смирнов Андрей	1	85 000	1
9	Смирнов Андрей	5	67 000	2
10	Смирнов Андрей	10	56 000	3

Задание №2: использование агрегатных оконных функций

Листинг 2

```
-- Задание №2: Нарастающий итог продаж по месяцам
WITH MonthlyOrders AS (
    -- Сначала агрегируем продажи по месяцам
    SELECT
        DATE_TRUNC('month', created_at)::DATE AS order_month,
        SUM(total_amount) AS monthly_amount
    FROM customer_order
    GROUP BY order_month
)
SELECT
    order_month AS "Месяц",
    monthly_amount AS "Сумма за месяц",
    -- Оконная функция считает нарастающий итог от первого месяца до текущего
    SUM(monthly_amount) OVER (ORDER BY order_month) AS "Нарастающий итог"
FROM MonthlyOrders
ORDER BY order_month;
```

	Месяц	Сумма за месяц	Нарастающий итог
1	2025-01-01	250 000	250 000
2	2025-02-01	376 000	626 000
3	2025-03-01	271 000	897 000

Задание №3: использование функции смещения

Листинг 3

```
-- Задание №3: Сравнение продаж с предыдущим месяцем
WITH TotalMonthlySales AS (
    SELECT
        DATE_TRUNC('month', created_at)::DATE AS sale_month,
        SUM(total_amount) AS total_amount
    FROM customer_order
    GROUP BY sale_month
)
SELECT
    sale_month AS "Месяц",
    total_amount AS "Продажи за текущий месяц",
    -- Получаем значение из предыдущей строки (смещение 1), если нет -
подставляем 0
    LAG(total_amount, 1, 0) OVER (ORDER BY sale_month) AS "Продажи за
предыдущий месяц",
    total_amount - LAG(total_amount, 1, 0) OVER (ORDER BY sale_month) AS
"Изменение"
FROM TotalMonthlySales
ORDER BY sale_month;
```

	Месяц	Продажи за текущий месяц	Продажи за предыдущий месяц	Изменение
1	2025-01-01	250 000	0	250 000
2	2025-02-01	376 000	250 000	126 000
3	2025-03-01	271 000	376 000	-105 000

Задание №4: построение сводной таблицы

Листинг 4

```
-- Задание №4: Сводная таблица продаж туроператоров по кварталам
SELECT
    t_op.name AS "Туроператор",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 1 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q1",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 2 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q2",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 3 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q3",
    SUM(CASE WHEN EXTRACT(QUARTER FROM co.created_at) = 4 THEN oi.price *
oi.quantity ELSE 0 END) AS "Q4"
FROM customer_order co
JOIN order_item oi ON co.id = oi.order_id
JOIN service s ON oi.service_id = s.id
JOIN tour_operator t_op ON s.tour_operator_id = t_op.id
GROUP BY t_op.name
ORDER BY t_op.name;
```

	A-Z Туроператор	123 Q1	123 Q2	123 Q3	123 Q4
1	ANEX Tour	210 000	0	0	0
2	Biblio Globus	32 000	0	0	0
3	Coral Travel	85 000	0	0	0
4	DSBW	183 000	0	0	0
5	Intourist	99 000	0	0	0
6	Mouzenidis Travel	183 000	0	0	0
7	Pegas Touristik	67 000	0	0	0
8	Sunmar	45 000	0	0	0
9	Tez Tour	176 000	0	0	0
10	Русский Экспресс	56 000	0	0	0

Задание №5: создание модифицируемого представления

Листинг 5

```
-- Задание №5: Создание модифицируемого представления для 5-звездочных отелей
CREATE OR REPLACE VIEW five_star_hotels AS
SELECT
    id,
    name,
    country_id,
    city_id,
    stars,
    full_address,
    description,
    meal_type_id
FROM hotel
WHERE stars = 5;
```

Задание №6: модификация данных через представление

Листинг 6

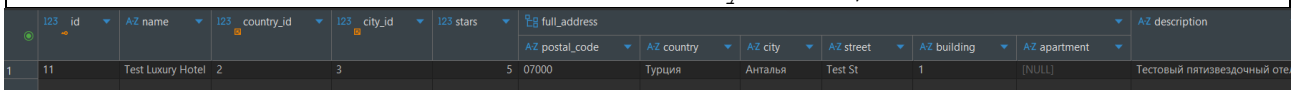
```
-- Задание №6: Модификация данных через представление

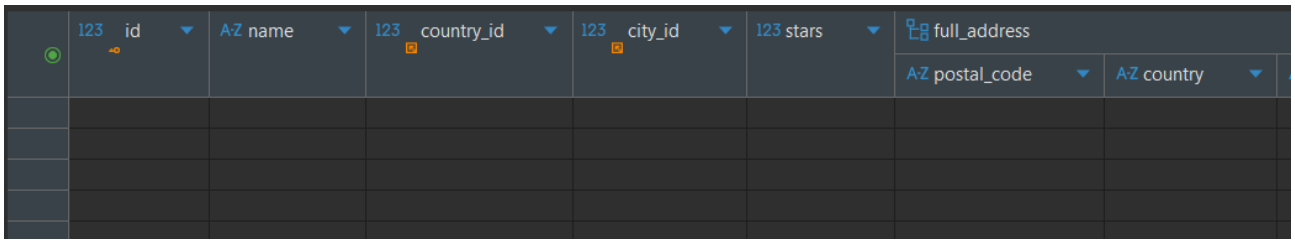
-- 1. Добавляем новую запись через представление
INSERT INTO five_star_hotels (name, country_id, city_id, stars, full_address,
description, meal_type_id)
VALUES (
    'Test Luxury Hotel',
    (SELECT id FROM country WHERE code = 'TR'),
    (SELECT id FROM city WHERE iata_code = 'AYT'),
    5,
    ROW('07000', 'Турция', 'Анталья', 'Test St', '1', NULL), -- Составной тип
address_type
    'Тестовый пятизвездочный отель',
    (SELECT id FROM meal_type WHERE code = 'AI')
);

-- Проверяем, что запись добавилась в базовую таблицу
SELECT * FROM hotel WHERE name = 'Test Luxury Hotel';

-- 2. Удаляем запись через представление
DELETE FROM five_star_hotels WHERE name = 'Test Luxury Hotel';

-- Проверяем, что запись удалилась из базовой таблицы
SELECT * FROM hotel WHERE name = 'Test Luxury Hotel';
```





Задание №7: создание немодифицируемого аналитического представления

Листинг 7

```
-- Задание №7: Немодифицируемое аналитическое представление по туроператорам
CREATE OR REPLACE VIEW tour_operator_analytics AS
SELECT
    t_op.name AS tour_operator_name,
    COUNT(DISTINCT s.id) AS total_services,
    COALESCE(SUM(oi.price * oi.quantity), 0) AS total_revenue,
    -- Условная агрегация: считаем только полностью оплаченные заказы
    COUNT(DISTINCT co.id) FILTER (WHERE os.name = 'Оплачен полностью') AS
paid_orders_count
FROM tour_operator t_op
LEFT JOIN service s ON t_op.id = s.tour_operator_id
LEFT JOIN order_item oi ON s.id = oi.service_id
LEFT JOIN customer_order co ON oi.order_id = co.id
LEFT JOIN order_status os ON co.status_id = os.id
```

```
GROUP BY t_op.name;
```

Задание №8: использование аналитического представления в запросах

Листинг 8

```
-- Задание №8: Фильтрация данных аналитического представления
SELECT
    tour_operator_name AS "Туроператор",
    total_revenue AS "Общая выручка",
    total_services AS "Кол-во услуг",
    paid_orders_count AS "Оплаченные заказы"
FROM tour_operator_analytics
WHERE total_revenue > 100000 -- Отбираем только успешных туроператоров
ORDER BY total_revenue DESC;
```

	Туроператор	Общая выручка	Кол-во услуг	Оплаченные заказы
1	ANEX Tour	210 000	1	0
2	DSBW	183 000	1	0
3	Mouzenidis Travel	183 000	1	0
4	Tez Tour	176 000	1	0

Задание №9: Создание и обновление материализованного представления

Листинг 9

```
-- Задание №9: Материализованное представление для трат клиентов

-- 1. Создаем материализованное представление
CREATE MATERIALIZED VIEW client_total_spent AS
SELECT
    c.id AS client_id,
    (c.full_name).last_name || ' ' || (c.full_name).first_name AS client_name,
    COUNT(co.id) AS orders_count,
    COALESCE(SUM(co.total_amount), 0) AS total_spent
FROM client c
LEFT JOIN customer_order co ON c.id = co.client_id
GROUP BY c.id, c.full_name;

-- Проверяем исходные данные
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';

-- 2. Добавляем новый заказ для клиента Иванов И.И. в исходную таблицу
INSERT INTO customer_order (created_at, client_id, employee_id, status_id,
paid_at, total_amount)
VALUES (
    CURRENT_TIMESTAMP,
    (SELECT id FROM client WHERE login = 'ivan_i'),
    (SELECT id FROM employee WHERE login = 'a_smirnov'),
    (SELECT id FROM order_status WHERE name = 'Новый'),
    NULL,
    50000.00
);
```

```sql
-- 3. Проверяем материализованное представление (данные еще НЕ обновились, так как оно хранит снимок)
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';

-- 4. Выполняем ручное обновление материализованного представления
REFRESH MATERIALIZED VIEW client_total_spent;

-- 5. Снова проверяем (теперь сумма total_spent увеличилась на 50000)
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';

-- --- Очистка тестовых данных (чтобы не засорять БД) ---
DELETE FROM customer_order
WHERE total_amount = 50000.00
  AND client_id = (SELECT id FROM client WHERE login = 'ivan_i')
  AND created_at >= CURRENT_DATE;
REFRESH MATERIALIZED VIEW client_total_spent;

-- 6. Проверяем удаление
SELECT * FROM client_total_spent WHERE client_name LIKE 'Иванов%';
```

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	1	85 000

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	1	85 000

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	2	135 000

	123 client_id	A-Z client_name	123 orders_count	123 total_spent
1	1	Иванов Иван	1	85 000